CSE 314 — Operating System Sessional
Online on xv6 Scheduling · Section C1

Strict Priority with Starvation Rescue

35 minutes. Fresh xv6 checkout, CPUS := 1.

The Idea

Replace the round-robin scheduler with strict priority, plus a rescue rule that lets processes run even while higher-priority processes remain runnable.

- Every process starts with wait counter 0. Before each scheduling decision, increment every RUNNABLE process's counter by exactly one, regardless of priority.
- If any RUNNABLE process has counter >= RESCUE_THRESH, rescue the one with the largest counter, regardless of priority. Break equal-counter ties by process-table order.
- Otherwise, pick the RUNNABLE process with the highest priority. Break equal-priority ties by process-table order.
- Reset the selected process's counter to 0, whichever rule selected it. Run it until it yields, blocks, or exits; repeat the procedure at the next scheduling decision.

Non-RUNNABLE processes keep their counters unchanged; sleeping does not increase them. If none are runnable, wait for work. Counters count scheduling decisions, not timer ticks. No timer-handler changes or explicit queues are required.

Specification Parameters

Define the following macros in kernel/param.h:

Macro	Value	Meaning
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	5	Priority every process starts with.
`RESCUE_THRESH`	20	Counter value that makes a process eligible for rescue.

System Calls

```
int setpriority(int priority);
int getpriority(void);
```

setpriority sets the caller's priority and returns 0. Invalid input resets priority to DEFAULT_PRIORITY and returns −1. Neither case resets the wait counter. getpriority returns the current priority. A fork() child inherits its parent's priority but starts with counter 0.

The syscall bodies are supplied in syspriority.c; append them to kernel/sysproc.c. Add priority and wait-counter fields to struct proc, initialize them on allocation and fork, and wire the calls through kernel/syscall.h, kernel/syscall.c, user/usys.pl and user/user.h using unused syscall numbers.

Logging

Keep a toggleable log line for every rescue, showing the counter before resetting it — a global flag or #define that you flip and rebuild is fine:

```
RESCUE: Process 4 (testloop) chosen with wait 20 at priority 2
```

Test Program

Use the attached testloop.c as-is: save it to user/testloop.c and add $U/_testloop to UPROGS in the Makefile. Calling syntax: testloop <iterations> <priority>.

Sample Session

Three testloop processes, equal work, launched lowest priority first, one line at a time (scheduler logging off). The & runs each process in the background so the next can be launched while it is still running:

```
$ testloop 60 2 &
PID 4: starting 60 iterations at time 7, priority 2
$ testloop 60 4 &
PID 6: starting 60 iterations at time 13, priority 4
$ testloop 60 8 &
PID 8: starting 60 iterations at time 19, priority 8
PID 8: finished at time 65
PID 6: finished at time 98
PID 4: finished at time 129
```

This session was captured under QEMU with CPUS := 1 using the supplied testloop.c and a reference implementation. PIDs, timestamps and completion order vary; exact timings are not part of evaluation. Finishing order alone does not demonstrate rescue: enable the log to observe rescue selections. Equal priorities do not share the CPU: the earlier table slot wins every priority race.

For a deterministic check, suppose only P1 and P2 are continuously runnable, in that table order, with priorities 8 and 2 and initial counters 0. Decisions 1–19 select P1. At decision 20, P2’s counter reaches 20 and P2 is rescued. Decision 21 selects P1 again. Reaching the threshold makes a process eligible for rescue; it may wait longer if others have larger counters.

Submission

Run these commands from the xv6 checkout. Replace STUDENT_ID with your student ID and submit the patch containing your scheduling changes and test program.

```
git add --all
git diff HEAD > ../STUDENT_ID.patch
```